

# Desarrollo de una celda reconfigurable de CGRA

Duarte Sanchez David, Jimenez Ulate Eddy, Ruiz Rivera Jeffrey, Esquivel Arias Fabian Zúñiga Ramírez Jose Eduardo

Grupo de 5

{davidduarte28, e.jimenez.10, yr1ruiz, fabiesqui22, je.zuniga}@estudiantec.cr

## I. INTRODUCCIÓN

Las arquitecturas reconfigurables han sido muy exitosas en la industria debido a que la evolución del software ha presentado una rápida evolución, con diversas aplicaciones, distintas necesidades de usuario y progreso científico, por lo que el hardware no se puede adaptar en la misma velocidad al software [1]. Sistemas de propósito específico pueden sufrir de obsolescencia rápida y si el precio del desarrollo y producción del hardware es alto, se vuelve inviable económicamente [1].

Actualmente, arquitecturas como las FPGA (*Field Programmable Gate Arrays*) han sido ampliamente adoptadas en diversas aplicaciones debido a su flexibilidad y capacidad de personalización, tomando fuerza en Computación de alto desempeño (HPC) por sus siglas en inglés. Sin embargo, las FPGAs presentan ciertas limitaciones como tiempos de configuración lentos, largos tiempos de compilación y aunque es algo relativo, bajas frecuencias de operación [2].

Debido a lo anterior, se ha tomado un interés en HPC en utilizar arquitecturas como los CGRAs (*Coarse-Grained Reconfigurable Arrays*) que ofrecen una solución intermedia entre los FPGAs y los procesadores de propósito general (GPP). Los CGRAs son arquitecturas de hardware que conceptualmente permiten que el silicio sea maleable y su funcionalidad pueda reconfigurarse de manera dinámica [2]. Los CGRA tienen la ventaja de ser uno o dos ordenes de magnitud más eficientes que las FPGAs y más de 2 a 3 ordenes de magnitud más eficientes que los GPPs [1].

El desarrollo de una celda reconfigurable de CGRA es de gran importancia por su relevancia no solo en HPC, si no también en su adopción en aplicaciones de *Machine-learning*, procesamiento de imágenes y visión por computadora, comunicaciones y otras áreas [3]. Por lo tanto el objetivo de este trabajo es el desarrollo de un CGRA heterogéneo, que posea tentativamente celdas de ruteo de datos, celdas con bancos de memoria distribuida, celdas de procesamiento escalar y celdas de procesamiento vectorial, con la capacidad de ejecutar programas utilizando el set de instrucciones RISC-V, con la capacidad de ejecutar lógica de ocho, 16 y 32 bits.

La importancia de utilizar este set de instrucciones es que este es un estándar abierto, facilitando su adopción debido a que no requiere pagos por licencias. Este es un set de instrucciones basado en la simplicidad, haciendo que el diseño de hardware sea sencillo y una implementación más directa [4].

## II. ANTECEDENTES Y TRABAJO RELACIONADO

En los últimos años, las CGRA han ganado un protagonismo significativo como aceleradores de hardware debido a su capacidad para ofrecer un equilibrio excepcional entre la flexibilidad del software y el alto rendimiento del hardware de aplicación específica [1], [2]. A diferencia de las arquitecturas de grano fino, las CGRAs operan a nivel de palabra o bloque, lo que reduce sustancialmente la sobrecarga de reconfiguración y enrutamiento, haciéndolas particularmente atractivas para aplicaciones eficientes en el borde (*edge computing*). El desarrollo e investigación de estas arquitecturas ha sido fuertemente impulsado por la adopción del repertorio de instrucciones (ISA) de código abierto RISC-V [4], cuya naturaleza extensible facilita la creación de Sistemas en Chip (SoC) heterogéneos altamente personalizados.

Investigaciones recientes han demostrado las ventajas de integrar estrechamente núcleos RISC-V con arreglos CGRA para superar el cuello de botella tradicional de Von Neumann. Por ejemplo, [5] exploran la integración de un acelerador CGRA acoplado a un núcleo de alto rendimiento RISC-V CVA6 enfocado en ecosistemas cloud-edge, mientras que [3] proponen DVHetero, un entorno de trabajo integral para el diseño y la validación de SoCs heterogéneos basados en esta combinación.

A nivel de implementación práctica, estas arquitecturas se materializan en esfuerzos de código abierto de vanguardia; repositorios como *risc-v-cgra* (desarrollado por ECASLab) investigan estas integraciones a nivel de sistema, mientras que proyectos como OpenEdgeCGRA (ESL-EPFL) proporcionan plataformas open-hardware completas. En este último, la arquitectura se compone de una malla bidimensional configurable de Elementos de Procesamiento (PEs) o celdas reconfigurables que interactúan en forma de toroide y comparten registros para maximizar la paralelización de instrucciones con una sobrecarga de memoria mínima. Otro proyecto destacable como OpenCGRA desarrollado por el PNNL (*Pacific Northwest National Laboratory*) el cual genera procesadores reconfigurables de forma parametrizable, de esta manera, permite diseñar el hardware por código como el tamaño de la malla, el tipo de celdas, cómo se comunican entre ellas, luego se prueba el funcionamiento usando Python y Verilator y finalmente se exporta el diseño a un código de diseño de hardware estándar (HDL) como Verilog.

Para que este tipo de sistemas alcance su máximo potencial, el diseño interno de la celda reconfigurable individual (PE) y

las herramientas de compilación asociadas son críticos [6]. Destacan la necesidad de un mapeo consciente del contexto de memoria (context-memory aware mapping) para asegurar una aceleración eficiente desde el punto de vista energético, subrayando que la latencia en el acceso a datos dicta el rendimiento global. En este marco, el diseño y desarrollo de una nueva celda reconfigurable de CGRA se fundamenta en la necesidad de optimizar las interconexiones locales, el acceso determinista a operandos intermedios y la versatilidad de sus operaciones a nivel de ciclo de reloj, retos que continúan siendo áreas de activa exploración en la frontera del diseño de aceleradores heterogéneos.

#### DECLARACIÓN DE USO DE HERRAMIENTAS DE IA

Para este trabajo se hizo uso de Inteligencia Artificial para la ayuda en la estructura de la introducción y el estado del arte, así como para una revisión gramatical y ortográfica del documento.

#### REFERENCIAS

- [1] L. Liu, J. Zhu, Z. Li, Y. Lu, Y. Deng, J. H. Han, S. Yin, and S. Wei, "A survey of coarse-grained reconfigurable architecture and design: Taxonomy, challenges, and applications," *ACM Computing Surveys*, vol. 52, pp. 1 – 39, 2019.
- [2] K. S. Artur Podobas and S. Matsuoka, "A survey on coarse-grained reconfigurable architectures from a performance perspective," *IEEE ACCESS*, vol. 8, 2020.
- [3] G. Zhu, L. Deng, K. Zhang, W. Fan, B. Jin, W. Cao, F. Zhang, X. Zhou, F. Zhang, and X. Yu, "Dvhetero: A framework for designing and validating heterogeneous soc with risc-v processor and cgra," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 18, pp. 1 – 30, 2025. [Online]. Available: <https://www.semanticscholar.org/paper/8d8821ac4c9ec42fc45e662ff65a0ccd6361297a>
- [4] EdgeIR Staff. (2024, jan) What is risc-v and why is it important? online. Accessed: 2026-06-11. [Online]. Available: <https://www.edgeir.com/what-is-risc-v-and-why-is-it-important-20240109>
- [5] J. Granja, D. Vázquez, A. Rodríguez, and A. Otero, "Integration of a cgra accelerator with a cva6 risc-v core for the cloud-edge," null. [Online]. Available: <https://www.semanticscholar.org/paper/b4dfe9170dd1bfa04c4eee19794ebe685b388fe0>
- [6] S. Das, K. J. M. Martin, and P. Coussy, "Context-memory aware mapping for energy efficient acceleration with cgras," *Design, Automation & Test in Europe Conference & Exhibition*, vol. null, pp. 336–341, 2019. [Online]. Available: <https://www.semanticscholar.org/paper/b3c002d29c59700cc1827aff29a72b560c86346a>